DOCUMENT 19 — API INTEGRATION, DATA SYNCHRONIZATION & STATE MANAGEMENT SPECIFICATION

1. Dokümanın Amacı

Bu dokümanın amacı Smart Chef & Pantry AI Flutter uygulaması ile Flask REST API arasındaki veri iletişiminin, state management yapısının ve server-client senkronizasyonunun standartlarını belirlemektir.

Bu doküman özellikle aşağıdaki konuları kapsar:

- Flutter ↔ Flask API iletişimi
- Dio kullanım standardı
- API request/response akışı
- Repository ve datasource ilişkisi
- Riverpod state management
- Server state ve local UI state ayrımı
- Loading/success/error/empty durumları
- Authentication state
- Favorites synchronization
- Cart synchronization
- Recipe recommendation state
- Cache yaklaşımı
- Retry ve timeout
- Optimistic UI
- Stale data yönetimi
- Login sonrası state yenileme
- Logout sonrası state temizleme
- Concurrent request yönetimi
- Data consistency
- API contract uyumu
- Integration testing

---

2. Temel Mimari Prensip

Flutter uygulaması backend ile doğrudan UI seviyesinde iletişim kurmayacaktır.

Doğru akış:

UI
↓
Provider / StateNotifier
↓
Use Case
↓
Repository Interface
↓
Repository Implementation
↓
Remote Data Source
↓
Dio
↓
Flask REST API

Geri dönüş:

Flask API
↓
Dio
↓
Remote Data Source
↓
DTO / Model
↓
Repository
↓
Domain Entity
↓
Use Case
↓
Provider
↓
UI

Bu yapı Document 07'de tanımlanan Clean Architecture'ın API entegrasyonundaki uygulamasıdır.

---

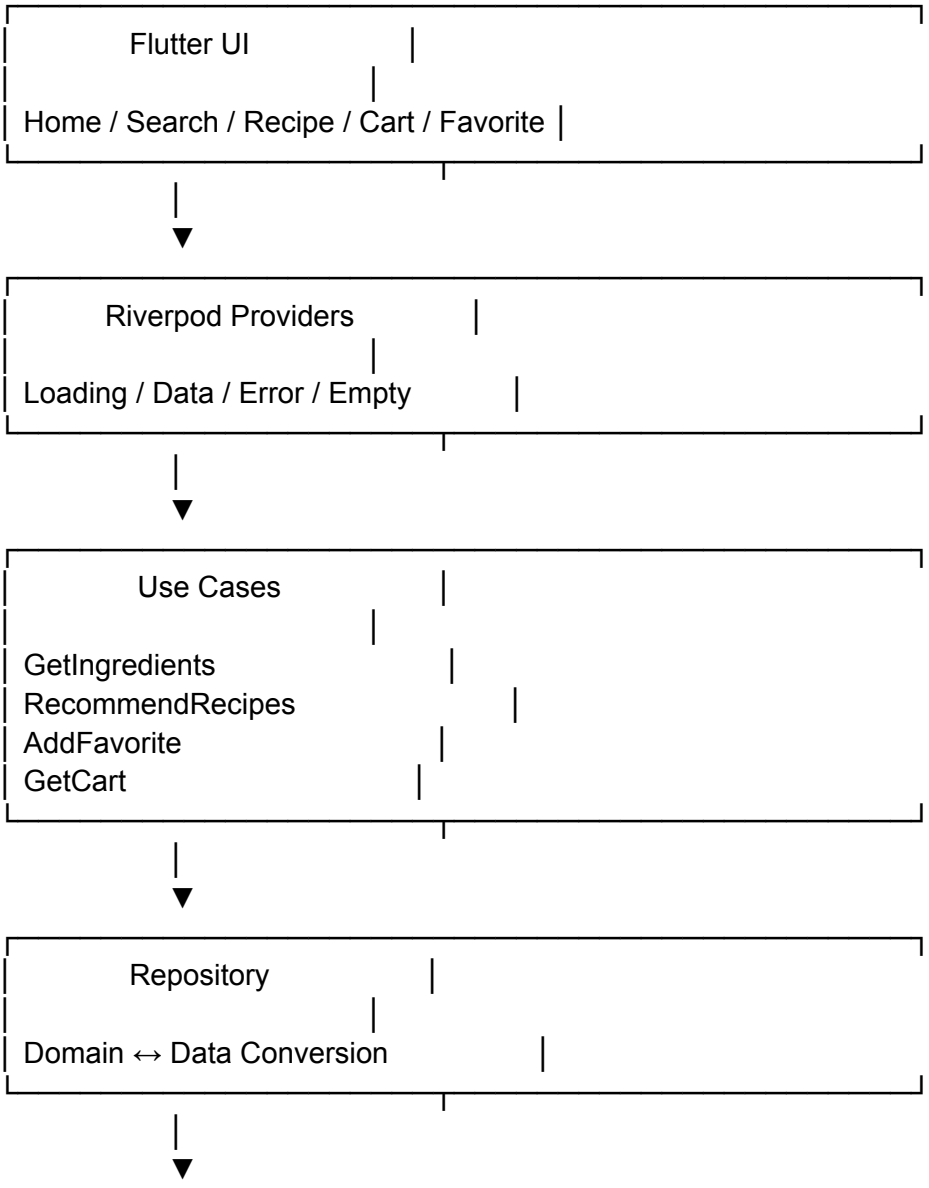
3. Veri Akışının Ana Kuralları

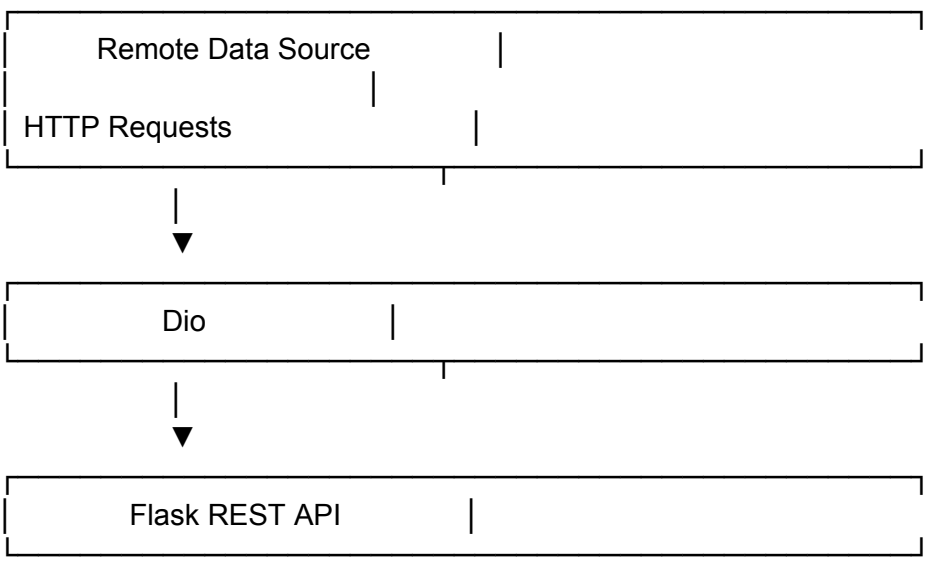
Aşağıdaki kurallar zorunludur:

1. UI doğrudan Dio kullanmaz.
2. Domain katmanı Dio bilmez.
3. Repository interface domain'de bulunur.
4. API modelleri data katmanında tutulur.
5. Domain entity'leri UI tarafından kullanılabilir.
6. Backend response doğrudan UI'a aktarılmaz.
7. API error'ları standart Failure yapısına dönüştürülür.
8. Authentication merkezi olarak yönetilir.
9. Server state ile local UI state birbirinden ayrılır.
10. Backend source of truth olarak kabul edilir.

---

4. API Integration Architecture





---

5. Dio Client

Uygulamada merkezi bir Dio client kullanılacaktır.

Önerilen dosya:

lib/core/network/dio_client.dart

Bu client aşağıdaki sorumlulukları taşıyabilir:

Base URL
Connect timeout
Receive timeout
Send timeout
Default headers
JSON handling
Interceptors

Örnek kavramsal yapı:

```
class DioClient {
  final Dio dio;

  DioClient(this.dio);
}
```

Dio instance uygulama genelinde kontrollü şekilde kullanılmalıdır.

---

6. Base URL Yönetimi

API URL doğrudan her datasource içinde yazılmamalıdır.

Yanlış:

dio.get("https://api.example.com/api/ingredients");

Doğru:

App Configuration
↓
Base URL
↓
Dio Client
↓
Endpoints

Örneğin:

Development
http://10.0.2.2:5000/api

Testing
https://test-api.example.com/api

Production
https://api.example.com/api

Environment'a göre değiştirilmelidir.

---

7. Android Emulator ve Local Backend

Android emulator üzerinde:

localhost

Flutter cihazının kendisini ifade eder.

Bilgisayardaki backend'e erişmek için Android emulator'da çoğu durumda:

10.0.2.2

kullanılması gerekir.

Örneğin:

http://10.0.2.2:5000/api

Development ortamına özel bir yapılandırma olarak tutulmalıdır.

Gerçek cihazda ise bilgisayarın local network IP adresi gerekebilir.

---

8. Environment Configuration

Flutter'da:

development
testing
production

ortamları birbirinden ayrılmalıdır.

Örneğin:

lib/core/constants/api_constants.dart

içerisinde endpoint path'leri tutulabilir.

Environment bilgisi ise ayrı configuration mekanizmasıyla yönetilmelidir.

Amaç:

Production URL

ile development URL'sinin karışmasını önlemektir.

---

9. Endpoint Constants

Endpoint path'leri uygulamanın farklı yerlerinde tekrar tekrar yazılmamalıdır.

Örnek:

```
class ApiEndpoints {
  static const socialLogin = '/auth/social';
  static const me = '/me';

  static const ingredients = '/ingredients';

  static const recommendRecipes = '/recipes/recommend';

  static const favorites = '/favorites';

  static const cart = '/cart';
  static const cartItems = '/cart/items';
}
```

Bu yapı typo kaynaklı hataları azaltır.

---

10. Dio Interceptor

Authentication için Dio interceptor kullanılacaktır.

Akış:

```
API Request
    ↓
Auth Interceptor
    ↓
JWT var mı?
    ↓
Authorization Header
    ↓
Flask
```

Header:

```
Authorization: Bearer <JWT>
```

şeklindedir.

---

11. Public Endpointler

JWT gerektirmeyen endpointler:

POST /api/auth/social
GET /api/ingredients
POST /api/recipes/recommend
GET /api/recipes/{recipe_id}

olarak belirlenmiştir.

Interceptor token eklese bile backend public endpointlerde token zorunlu tutmayabilir.

---

12. Protected Endpointler

JWT gerektiren endpointler:

GET /api/me

GET /api/favorites
POST /api/favorites
DELETE /api/favorites/{recipe_id}

GET /api/cart
POST /api/cart/items
PATCH /api/cart/items/{cart_item_id}
DELETE /api/cart/items/{cart_item_id}
DELETE /api/cart

şeklindedir.

---

13. Authentication State

Flutter tarafında merkezi authentication state bulunmalıdır.

Temel durumlar:

Unauthenticated
Authenticating
Authenticated
AuthError

Akış:

App Start
↓
Check Token
↓
Token Exists?
┌──────────┴──────────┐
No Yes
↓ ↓
Unauthenticated Validate / Restore
↓
Authenticated

---

14. Application Startup

Uygulama açıldığında:

main.dart
↓
Initialize Dependencies
↓
Initialize Secure Storage
↓
Initialize Auth Provider
↓
Check Existing JWT
↓
Load /api/me if required
↓
Determine Auth State
↓
Start Router

şeklinde ilerleyebilir.

UI'ın authentication state kesinleşmeden yanlış ekran göstermesi engellenmelidir.

---

15. Router ve Authentication

go_router authentication state ile birlikte çalışmalıdır.

Örneğin:

Guest
↓
/home

erişebilir.

Ancak:

/favorites
/cart
/profile

gibi protected ekranlara gitmeye çalışırsa:

Login

ekranına yönlendirilmelidir.

---

16. Return Route

Kullanıcı:

Recipe Detail
↓
Favorite

yaptığında login gerekiyorsa:

Recipe Detail
↓
Login

↓
Google
↓
Authenticated
↓
Return Recipe Detail
↓
Continue Favorite

akışı korunmalıdır.

Bu davranış Document 10'da tanımlanan authentication flow ile uyumlu olmalıdır.

---

17. API Response Parsing

Backend standardı:

```
{
  "success": true,
  "data": {},
  "message": "..."
}
```

olduğu için Flutter response parser bu yapıya uygun olmalıdır.

Örneğin:

Dio Response
↓
API Envelope
↓
data
↓
DTO

ayrıştırması yapılabilir.

UI doğrudan:

response.data["data"]

şeklinde işlem yapmamalıdır.

---

18. DTO ve Domain Entity Ayrımı

Örneğin backend:

```
{
  "id": "...",
  "title": "...",
  "match_percentage": 80
}
```

döndürür.

Flutter data layer:

RecipeRecommendationModel

oluşturur.

Domain:

RecipeRecommendation

entity'sini kullanır.

Akış:

JSON
↓
Model
↓
Entity
↓
UI

olmalıdır.

---

19. JSON Naming

Backend:

snake_case

kullanır.

Örneğin:

```
{
  "match_percentage": 80,
  "image_url": "...",
  "missing_ingredients": []
}
```

Flutter Dart model:

```
@JsonKey(name: 'match_percentage')
final double matchPercentage;
```

şeklinde map edilebilir.

Böylece backend ve Flutter naming convention'ları birbirine zorla dönüştürülmeden uyumlu hale gelir.

---

20. Request Model

API requestleri de model olarak tanımlanmalıdır.

Örneğin:

RecipeRecommendationRequest

içerisinde:

ingredient_ids

bulunur.

Flutter:

Entity / Input
↓
Request Model
↓

JSON
↓
Dio

şeklinde çalışmalıdır.

---

21. Repository Responsibility

Repository:

Domain ↔ Data

köprüsüdür.

Örneğin:

RecipeRepository

sorumlulukları:

Get recipe detail
Recommend recipes
Convert model → entity
Handle datasource errors

olabilir.

Repository business rule'ların ana sahibi değildir.

Recommendation scoring backend'dedir.

---

22. Use Case Responsibility

Use case belirli bir application action'ı temsil eder.

Örneğin:

GetIngredients
RecommendRecipes

GetRecipeDetail
AddFavorite
RemoveFavorite
GetFavorites
GetCart
AddCartItem
UpdateCartItem
RemoveCartItem

Use case:

UI Input
↓
Validation if appropriate
↓
Repository
↓
Result

akışını yönetir.

---

23. Provider Responsibility

Riverpod provider:

UI state
dependency injection
async state

gibi görevleri yönetir.

Provider'ın içine bütün business logic doldurulmamalıdır.

Yanlış:

Provider
├── HTTP
├── JSON parsing
├── Database rules
├── Recommendation calculation
└── UI state

Doğru:

Provider
↓
Use Case
↓
Repository
↓
Datasource

---

24. Async State Model

API kullanan ekranlarda temel state:

Initial
Loading
Success
Empty
Error

olarak ele alınmalıdır.

Örneğin recipe results:

Initial
↓
Loading
↓
Success

veya:

Loading
↓
Empty

veya:

Loading
↓
Error

olabilir.

---

25. Loading State

Loading sırasında:

CircularProgressIndicator
Skeleton
Shimmer

gibi UI kullanılabilir.

Ancak her küçük işlemde bütün ekranın loading olması gerekmeyebilir.

Örneğin cart quantity:

Tomato
[-] 2 [+]

değiştirirken yalnızca ilgili item loading olabilir.

---

26. Empty State

Empty state hata değildir.

Örneğin:

No recipes found.

API'nin başarılı şekilde:

200 OK

dönüp boş liste vermesi olabilir.

UI:

Success
+

Empty Data

durumunu ayrı göstermelidir.

---

27. Error State

API:

500

döndürdüğünde:

Error

state'i oluşturulmalıdır.

Kullanıcıya teknik exception yerine anlamlı mesaj gösterilmelidir.

Örneğin:

Something went wrong.
Please try again.

ve:

Retry

butonu.

---

28. Failure Mapping

Dio exception:

DioException

doğrudan UI'a gönderilmemelidir.

Akış:

DioException
↓
Exception Handler
↓
ApiException
↓
Failure
↓
Provider
↓
UI

Document 14 ile uyumlu olarak standartlaştırılmalıdır.

---

29. HTTP Status Mapping

Temel mapping:

200 → Success
201 → Created
204 → Success without body

400 → ValidationFailure
401 → AuthenticationFailure
403 → AuthorizationFailure
404 → NotFoundFailure
409 → ConflictFailure
422 → ValidationFailure
500 → ServerFailure
503 → ServiceUnavailableFailure

---

30. Network Error

Internet yoksa:

NoInternetFailure

oluşturulabilir.

UI:

No internet connection.
Please check your connection and try again.

gösterebilir.

Retry seçeneği sunulabilir.

---

31. Timeout Error

API timeout olduğunda:

TimeoutFailure

döndürülebilir.

UI:

The request took too long.
Please try again.

mesajı gösterebilir.

---

32. Retry

Retry yalnızca uygun durumlarda uygulanmalıdır.

Uygun örnekler:

GET ingredients
GET recipe detail
GET favorites
GET cart

Geçici network failure durumunda tekrar denenebilir.

Daha dikkatli olunması gerekenler:

POST favorite
POST cart item

Çünkü kontrolsüz retry duplicate action oluşturabilir.

Backend idempotency desteği gelecekte değerlendirilebilir.

---

33. Idempotency

Örneğin:

POST /api/favorites

aynı recipe için iki kez çağrılırsa database:

UNIQUE(user_id, recipe_id)

sayesinde duplicate kaydı engellemelidir.

Backend:

FAVORITE_ALREADY_EXISTS

gibi kontrollü response döndürebilir.

Bu, client retry kaynaklı sorunların etkisini azaltır.

---

34. Server State ve UI State

Bu ayrım kritik olacaktır.

Server state:

Recipes
Favorites
Cart
Ingredients
Profile

Backend'den gelir.

UI state:

Selected ingredient chips
Search text
Current tab
Dialog visibility
Loading indicator
Expanded card

gibi bilgilerden oluşur.

Server state ile UI state birbirine karıştırılmamalıdır.

---

35. Ingredient Selection State

Ingredient seçimi öncelikle local UI state olabilir.

Örneğin:

Selected:
Tomato
Onion
Potato

API çağrısı kullanıcı:

Find Recipes

butonuna bastığında yapılabilir.

Akış:

Select Ingredients
↓
Local State
↓
Find Recipes
↓
API
↓
Recipe Results

Her ingredient seçildiğinde recommendation request göndermek MVP için gerekli değildir.

---

36. Recipe Recommendation State

Recommendation ekranında:

selectedIngredientIds
recommendationResults
loading
error

gibi state bulunabilir.

Akış:

Ingredient Selection
↓
RecommendRecipes UseCase
↓
API
↓
Results
↓
Provider State
↓
Recipe List

---

37. Favorite State

Favorite işlemi server state'tir.

Akış:

Tap Favorite
↓
Authentication Check
↓
POST /favorites
↓

Backend
↓
Success
↓
Local UI State Update

Başarısız olursa UI server ile tutarlı hale getirilmelidir.

---

38. Optimistic Favorite UI

Favorite icon için optimistic UI kullanılabilir.

Örneğin:

Outline Heart
↓
Tap
↓
Filled Heart
↓
API Request

API başarısız olursa:

Filled Heart
↓
Rollback
↓
Outline Heart

yapılabilir.

Bu yaklaşım yalnızca hata rollback güvenilir şekilde uygulanabiliyorsa kullanılmalıdır.

---

39. Cart State

Cart backend source of truth'tur.

Örneğin:

Tomato × 2

quantity:

+

ile artırılır.

Akış:

UI
↓
UpdateCartItem
↓
API
↓
Server
↓
Updated Cart Item
↓
Provider State
↓
UI

---

40. Cart Optimistic Update

Cart quantity için optimistic update yapılabilir.

Ancak:

UI = 3
Server = 2

gibi senkronizasyon problemi oluşursa server response ile state düzeltilmelidir.

Daha güvenli yaklaşım:

UI action
↓
Temporary local update
↓
API
↓

Success → keep
Failure → rollback

---

41. Cart Concurrent Updates

Kullanıcı hızlı şekilde:

+ + + +

basarsa dört ayrı request oluşabilir.

Bu durum:

race condition

oluşturabilir.

MVP'de quantity update işlemlerinde:

request queue
debounce
latest-value strategy

gibi yaklaşımlar değerlendirilebilir.

Özellikle son değer önemliyse:

2 → 3 → 4 → 5

için yalnızca:

quantity = 5

requestinin gönderilmesi daha verimli olabilir.

---

42. Favorites Synchronization

Login sonrası:

GET /api/favorites

çağrılarak server state alınabilir.

Akış:

Login Success
↓
Load Favorites
↓
Favorites Provider
↓
UI

Logout olduğunda:

Favorites Provider
↓
Clear

yapılmalıdır.

---

43. Cart Synchronization

Aynı şekilde login sonrası:

GET /api/cart

çağrılabilir.

Logout:

Cart Provider
↓
Clear User Data

şeklinde olmalıdır.

Yeni kullanıcı login olduğunda önceki kullanıcının cart state'i ekranda kalmamalıdır.

---

44. User Switching

Özellikle geliştirme ve test sırasında:

User A logout
↓
User B login

olduğunda:

A favorites
A cart
A profile

state'leri temizlenmelidir.

Ardından:

B favorites
B cart
B profile

yüklenmelidir.

Bu veri izolasyonu açısından kritik bir konudur.

---

45. Logout Flow

Logout:

User
↓
Logout
↓
Optional backend logout endpoint
↓
Clear JWT
↓
Clear authenticated providers
↓
Clear private cached state
↓

Unauthenticated
↓
Login / Home

şeklinde ilerlemelidir.

Guest'a ait public recipe/ingredient state'leri gerekli ise korunabilir.

---

46. API Request Lifecycle

Her request:

UI Action
↓
Provider
↓
Use Case
↓
Repository
↓
Datasource
↓
Dio
↓
Interceptor
↓
HTTP
↓
Flask
↓
Response
↓
Dio
↓
Datasource Model
↓
Repository Entity
↓
Provider
↓
UI

akışını izlemelidir.

---

47. API Cancellation

Kullanıcı search ekranından çıkarsa devam eden search requestinin iptal edilmesi değerlendirilebilir.

Örneğin:

Search "tomato"
↓
Request
↓
User leaves screen
↓
Cancel request

Böylece gereksiz network ve state update azaltılabilir.

---

48. Stale Response Problemi

Örneğin kullanıcı:

Search A

sonra hemen:

Search B

yaparsa:

Request A
Request B

aynı anda çalışabilir.

Request A geç dönerse B sonucunun üzerine A sonucu yazılabilir.

Bu probleme karşı:

request identifier

cancel previous request
latest request wins

yaklaşımı kullanılmalıdır.

---

49. Latest Request Wins

Search gibi state'in son request tarafından belirlenmesi gereken işlemlerde:

Request 1 → "tomato"
Request 2 → "tomato sauce"

sonuçlarından yalnızca güncel request'in UI state'i değiştirmesine izin verilmelidir.

Bu özellikle:

Ingredient Search
Recipe Search

gibi alanlarda önemlidir.

---

50. Data Refresh

Kullanıcı:

Favorites

ekranını açtığında server state'in ne zaman yenileneceği belirlenmelidir.

MVP yaklaşımı:

Screen open
↓
Load favorites

veya:

Pull to refresh

kullanılabilir.

Gereksiz sürekli polling yapılmayacaktır.

---

51. Pull to Refresh

Aşağıdaki ekranlarda uygulanabilir:

Favorites
Cart

Örneğin:

Pull
↓
GET /api/cart
↓
Update Provider

kullanıcıya güncel server state'i görme imkanı verir.

---

52. Cache + Server Refresh

Cache kullanılıyorsa:

Cached Data
↓
Show Quickly
↓
Background Refresh
↓
Server Data
↓
Update UI

yaklaşımı uygulanabilir.

Ancak MVP'de karmaşıklığı azaltmak için yalnızca gerekli ekranlarda kullanılmalıdır.

---

53. API Contract Versioning

MVP'de:

/api

base path kullanılmaktadır.

İleride breaking changes oluşursa:

/api/v1
/api/v2

gibi versioning değerlendirilebilir.

Ama her küçük değişiklikte yeni API version oluşturulmamalıdır.

---

54. Backward Compatibility

Backend değişirken Flutter'ın mevcut sürümü tamamen bozulmamalıdır.

Örneğin yeni alan eklemek:

```
{
  "title": "...",
  "new_field": "..."
}
```

genellikle eski client için daha güvenlidir.

Var olan alanı silmek veya anlamını değiştirmek breaking change oluşturabilir.

---

55. API Contract Testing

Document 15 ile uyumlu şekilde API contract testleri yapılmalıdır.

Kontrol:

Request Schema
Response Schema
Field Names
Field Types
Nullable Fields
HTTP Status
Error Codes

olmalıdır.

Özellikle:

match_percentage
missing_ingredients
matched_ingredients

alanlarının tipi değişmemelidir.

---

56. Serialization Rules

Tüm JSON modelleri:

snake_case

API standardına uygun olmalıdır.

Tarih:

ISO 8601

UUID:

String

decimal/quantity:

API contract'ta açıkça tanımlı format

olmalıdır.

Flutter model conversion sırasında type assumptions yapılmamalıdır.

---

57. Nullability

Backend nullable bir alan döndürebiliyorsa Flutter modelinde de bu durum açıkça tanımlanmalıdır.

Örneğin:

youtube_url: String?

olabilir.

UI:

youtube_url == null

durumunda YouTube butonunu göstermeyebilir.

Null alanları zorla empty string'e çevirmek her durumda doğru değildir.

---

58. API Error Details

UI için gerekli hata detayları:

code
message
field errors
request_id

gibi kontrollü alanlar olabilir.

Ancak:

SQL exception
stack trace
internal file path
database credentials

client'a gönderilmemelidir.

---

59. Form / Input Validation

Flutter validation kullanıcı deneyimi için uygulanır.

Örneğin:

Empty ingredient selection

için kullanıcı daha API'ye request göndermeden uyarılabilir.

Ancak backend de aynı kontrolü yapmalıdır.

Flutter validation
+
Backend validation

birbirinin alternatifi değildir.

---

60. API Integration Error Flow

Örnek:

User taps "Find Recipes"
↓
No ingredient selected?
↓
Local Validation
↓
Show message

Ingredient seçilmişse:

Find Recipes
↓
Provider Loading
↓

Use Case
↓
Repository
↓
Dio
↓
API

API:

200

dönerse:

Success

API:

422

dönerse:

ValidationFailure

API:

500

dönerse:

ServerFailure

---

61. Recipe Detail Integration

Akış:

Recipe Result Card
↓
Tap
↓

GET /api/recipes/{recipe_id}
↓
RecipeRepository
↓
RecipeDetailProvider
↓
Recipe Detail Screen

Recipe ID Flutter tarafından backend'e gönderilir.

Backend UUID'yi doğrular.

Recipe bulunamazsa:

404
RECIPE_NOT_FOUND

döner.

---

62. Missing Ingredients Integration

Recipe detail response içerisinde:

matched_ingredients
missing_ingredients

bulunabilir.

Flutter backend'in hesapladığı:

match_percentage

değerini yeniden hesaplamamalıdır.

Örneğin backend:

80%

döndürdüyse Flutter:

80%

göstermelidir.

---

63. Missing Ingredient → Cart

Kullanıcı:

Missing Ingredient
↓
Add to Cart

yaptığında:

POST /api/cart/items

kullanılır.

Request:

```
{
  "ingredient_id": "...",
  "quantity": 1,
  "unit": null
}
```

olabilir.

Backend merge kuralını uygular.

Flutter merge hesaplamasını tekrar yapmaz.

---

64. Favorite Integration

Recipe Detail:

Favorite

butonuna basıldığında:

Authenticated?

kontrol edilir.

Guest:

Login

ekranına yönlendirilir.

Authenticated:

POST /api/favorites

çağrılır.

Başarılı response sonrası:

Favorite Provider

güncellenir.

---

65. Cart Integration

Cart ekranı açıldığında:

GET /api/cart

çağrılır.

Response:

CartResponse

modeline dönüştürülür.

UI:

CartItem
Cart total / count
Quantity controls

gösterebilir.

---

66. Cart Quantity Update

Kullanıcı:

+

bastığında:

current quantity + 1

hesaplanır.

Ardından:

PATCH /api/cart/items/{id}

gönderilir.

Backend:

quantity > 0

kontrol eder.

Başarılı response:

Updated CartItem

döner.

Provider server response ile güncellenir.

---

67. Cart Remove

Remove:

DELETE /api/cart/items/{id}

kullanır.

Başarılı:

204

veya standart API response alınabilir.

Provider ilgili item'ı kaldırır.

---

68. Clear Cart

DELETE /api/cart

kullanılır.

Başarılı olduğunda:

CartProvider
↓
Empty

durumuna geçer.

UI:

Your cart is empty.

gösterebilir.

---

69. Favorites List

GET /api/favorites

sonucu:

Loading
↓
Success
↓
Favorite Recipes

şeklinde gösterilir.

Liste boşsa:

No favorite recipes yet.

gibi empty state kullanılabilir.

---

70. Authentication Failure During API Call

Örneğin cart request:

GET /api/cart

ve backend:

401 TOKEN_EXPIRED

döndürürse:

Dio Interceptor
↓
Auth Failure
↓
Clear JWT
↓
Auth State → Unauthenticated
↓
Protected screen handling

uygulanmalıdır.

Aynı token hatasının her provider tarafından ayrı ayrı ele alınmasından kaçınılmalıdır.

---

71. Global Authentication Handling

Token expiration gibi global olaylar merkezi olarak yönetilmelidir.

Yanlış:

CartProvider → token clear
FavoriteProvider → token clear
ProfileProvider → token clear

Doğru:

Dio Interceptor
↓
Auth Event
↓
Auth Provider
↓
Unauthenticated

şeklindedir.

---

72. Dependency Injection

Riverpod dependency injection için kullanılabilir.

Örneğin:

Dio Provider
↓
IngredientRemoteDataSource
↓
IngredientRepository
↓
GetIngredientsUseCase
↓
IngredientProvider

şeklinde dependency graph oluşturulabilir.

Bu yapı testlerde mock dependency kullanımını kolaylaştırır.

---

73. Testability

API integration kodu doğrudan global singletonlara bağımlı olmamalıdır.

Testte:

Mock Dio
Mock DataSource
Mock Repository

kullanılabilmelidir.

Örneğin:

Provider
↓
Mock UseCase
↓
Success

ile UI test edilebilir.

---

74. API Mocking

Flutter testlerinde gerçek backend'e bağımlı olmak yerine mock response kullanılabilir.

Örneğin:

```
{
  "success": true,
  "data": {
    "items": []
  }
}
```

ile empty state test edilebilir.

Error:

```
{
  "success": false,
  "error": {
    "code": "SERVER_ERROR",
    "message": "..."
  }
}
```

ile error state test edilebilir.

---

75. Integration Test

Gerçek backend ile:

Login
Ingredient Search
Recommendation
Recipe Detail
Favorite
Cart

akışları test edilmelidir.

Örnek:

Login
↓
Select Tomato
↓
Recommend
↓
Open Recipe
↓
Favorite
↓
Open Favorites
↓
Recipe exists

---

76. Data Consistency Rules

Aşağıdaki kurallar korunmalıdır:

Backend = Source of Truth
Flutter = Presentation + Local UI State
Recommendation score = Backend
Cart merge = Backend
Favorite uniqueness = Backend + DB
Authentication identity = JWT
Authorization = Backend

Flutter yalnızca UI deneyimini iyileştirmek için optimistic/local state kullanabilir.

---

77. Synchronization Rules

Login

JWT
↓
Auth State
↓
Load User
↓
Load Favorites
↓
Load Cart

Logout

Clear JWT
↓
Clear Private State
↓
Unauthenticated

Favorite Add

API Success
↓

Update Favorite State

Cart Update

API Success
↓
Update Cart State

---

78. Data Refresh Strategy

MVP:

App startup
↓
Auth restore

Screen entry
↓
Load current data

Pull-to-refresh
↓
Reload

Mutation success
↓
Update local state from server response

Sürekli polling kullanılmayacaktır.

---

79. API Integration Anti-Patterns

Aşağıdaki yaklaşımlar kullanılmayacaktır:

❌ Widget içinde Dio çağırmak
❌ UI'da JSON parsing yapmak
❌ DioException'ı UI'a göndermek
❌ Backend response'u doğrudan widget'a vermek
❌ Domain layer'da Dio import etmek

❌ Client-side recommendation score hesaplamak
❌ Client user_id'sine güvenmek
❌ Her request sonrası bütün app state'i yenilemek
❌ Her karakterde API request göndermek
❌ Kontrolsüz retry
❌ Logout sonrası private state'i bırakmak
❌ User A state'ini User B'ye taşımak

---

80. API Integration Checklist

Development tamamlandığında:

[ ] Dio merkezi yapılandırıldı
[ ] Base URL environment'a göre değişiyor
[ ] Endpoint constants mevcut
[ ] Auth interceptor mevcut
[ ] JWT secure storage'dan alınıyor
[ ] Public/protected endpoint ayrımı yapıldı
[ ] DTO modelleri oluşturuldu
[ ] Domain entity'leri ayrıldı
[ ] Repository abstraction mevcut
[ ] Use case'ler oluşturuldu
[ ] Riverpod state yönetimi mevcut
[ ] Loading state mevcut
[ ] Empty state mevcut
[ ] Error state mevcut
[ ] Retry davranışı belirlendi
[ ] Timeout belirlendi
[ ] Token expiration merkezi yönetiliyor
[ ] Logout state temizliyor
[ ] Favorites synchronization çalışıyor
[ ] Cart synchronization çalışıyor
[ ] Recommendation API entegrasyonu çalışıyor
[ ] API contract testleri mevcut

---

81. Definition of Integration Done

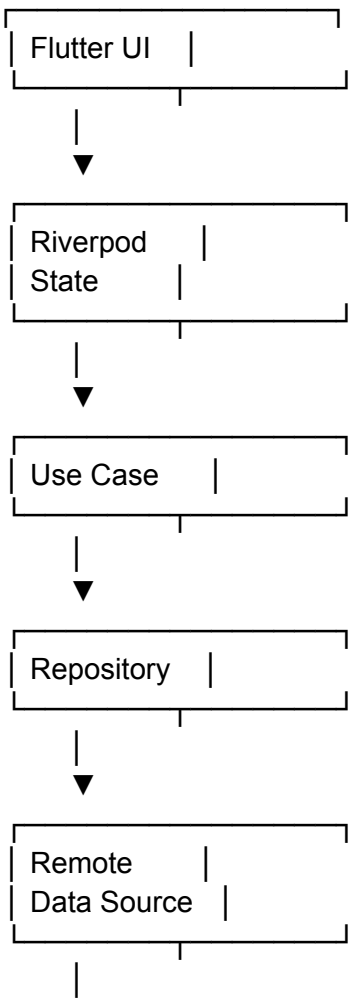
Bir feature API entegrasyonu açısından tamamlanmış sayılması için:

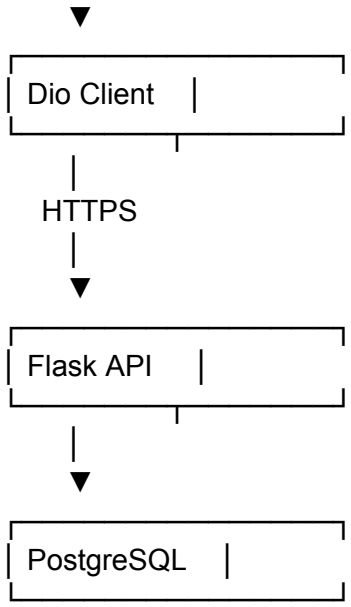
[ ] Endpoint backend'de hazır
[ ] Request model mevcut

[ ] Response model mevcut
[ ] JSON mapping doğru
[ ] Datasource implement edildi
[ ] Repository implement edildi
[ ] Use case implement edildi
[ ] Provider oluşturuldu
[ ] Loading state çalışıyor
[ ] Success state çalışıyor
[ ] Empty state çalışıyor
[ ] Error state çalışıyor
[ ] Authentication davranışı belirlendi
[ ] Retry/timeout davranışı belirlendi
[ ] Unit test mevcut
[ ] Integration test mevcut
[ ] UI ile gerçek endpoint test edildi

---

82. Nihai API & State Mimarisi





---

83. Sonuç

Smart Chef & Pantry AI'nin Flutter ve Flask arasındaki veri iletişimi Clean Architecture, merkezi API yönetimi ve kontrollü state management üzerine kurulacaktır.

Temel prensipler:

UI
↓
Riverpod
↓
Use Case
↓
Repository
↓
Data Source
↓
Dio
↓
Flask API
↓
PostgreSQL

şeklinde korunacaktır.

Server state ile local UI state birbirinden ayrılacak, backend uygulamanın veri açısından source of truth'u olacaktır.

Authentication merkezi olarak yönetilecek; JWT expiration, logout ve user switching gibi durumlarda kullanıcıya ait private state temizlenecektir.

Favorites ve cart gibi kullanıcıya özel veriler backend ile senkronize tutulacak; recipe recommendation sonuçları backend tarafından üretilecek ve Flutter yalnızca sonucu görüntüleyecektir.

Bu mimari sayesinde ileride:

Offline Cache
Redis
Background Sync
Refresh Token
Web Client
Advanced Recommendation
ML/NLP

gibi özelliklerin mevcut sistemi tamamen değiştirmeden eklenebilmesi hedeflenmektedir.